COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	08/28/2026	**DATE SUBMITTED:**	08/28/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 6 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

What procedures did you perform?

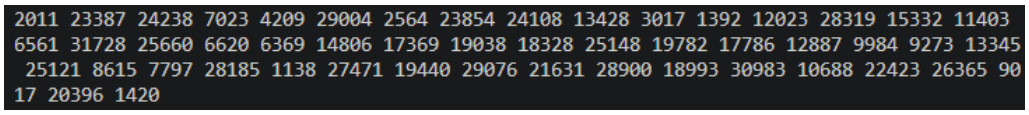
Screenshot	
Observations	Based on the data generated through the given code snippet, as represented in the screenshot above, with a constant value or size integer of 50, it randomly generated 50 random integers inside the dataset. As long as it is treated or a valid integer, it will be a possibility of generating inside the dataset. With this, it is treated as an unsorted value which is suited for linear searching algorithm technique since binary search algorithm technique is strictly limited for sorted values / dataset.

Table 6-1. Data Generated and Observations.

Code

```
31  // ------------------------------------------- //
32  /* LINEAR SEARCH ARRAY */
33
34  template <typename T>
35  void linearSearchArr(int maxSize, T dataFind, T data[]){
36      int i = 0;
37      while(i <= maxSize){
38          if(dataFind == data[i]){
39              std::cout << "Searching is successful\n";
40              return;
41          }
42          i++;
43      }
44      std::cout << "the search is unsuccessful\n";
45
46  }
```

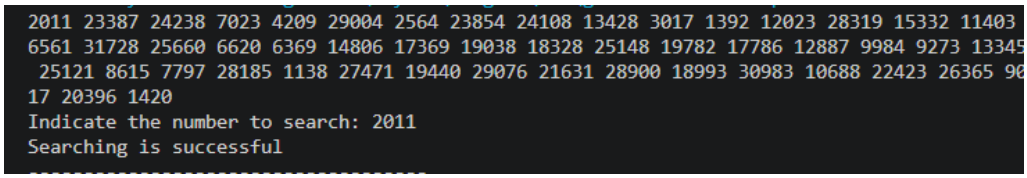
Output	
Observations	**In the line of code,** the linearSearchArr template function successfully implements a sequential search algorithm. It scans the array elements one by one starting from index 0. When the target value 2011 is found, it immediately prints "Searching is successful" and terminates early via the return; statement. The console output confirms that the algorithm correctly handles an array filled with unsorted integers. The program successfully identified that the input element 2011 exists within the dataset.

Table 6-2a. Linear Search for Arrays

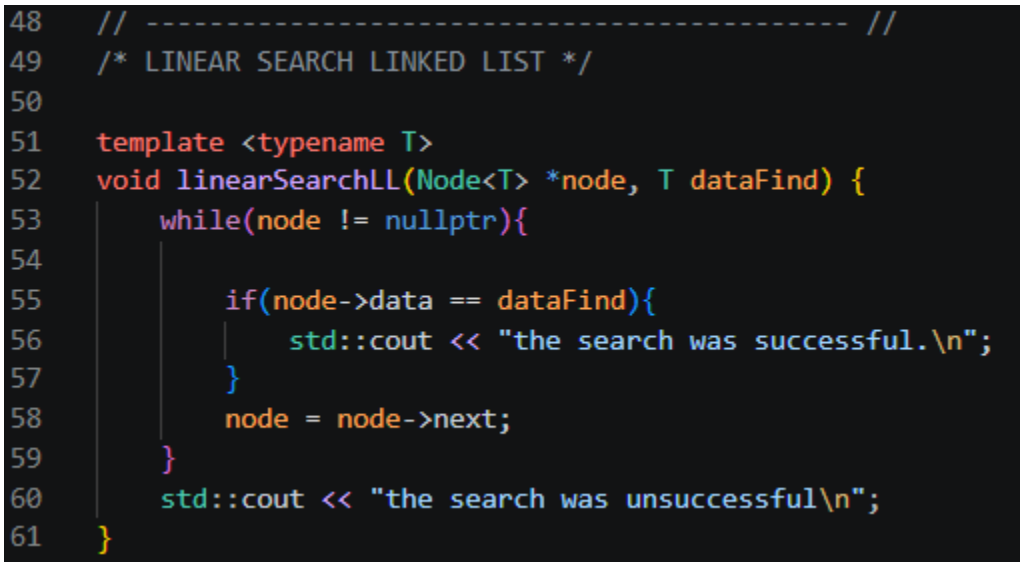
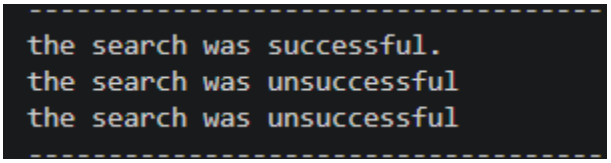
Code	
Output	
Observations	In this observed line of code, the function is designed to perform a **linear search on a singly linked list** to find a specific value dataFind. If the value exists in the list, it has the condition to notify the user of a successful search. If the value is missing after checking every node, its triggers the second condition of notifying the user that the search is unsuccessful.

Table6-2b. Linear Search for Linked List

Code

```
63  // ---------------------------------------- //
64  /* BINARY SEARCH ARRAY */
65
66  template <typename T>
67  void binarySearchArr(T dataFind, T data[], int maxSize){
68      int low = 0, up = maxSize-1, middle;
69      while(low <= up){
70          middle = (int)((low+up)/2);
71          if(dataFind == data[middle]){
72              std::cout << "\nSearching is successful\n";
73              return;
74          }
75          else if(dataFind > data[middle]){
76              low = middle + 1;
77          }else {
78              up = middle - 1;
79          }
80      }
81      std::cout << "Search element was not found.\n";
82  }
```

Output

```
-----------------------------------
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 3
3 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
Searching is successful
Search element was not found.
-----------------------------------
```

Observations

In this line of code, transitioning on the binary search array, The program's code **implements a Binary Search algorithm to locate a specific element within an array. But, for the binary search algorithm to be implemented,** the **array elements (0 to 49)** are arranged in a **sorted, ascending order**, for the binary search to function correctly.

As for the algorithm, it efficiently narrows down the search space by dynamically updating the **low** or **up** bounds based on a comparison with the calculated **middle** element. This binary search algorithm has a **time complexity** compared to linear searching, which checks elements sequentially, this binary search achieves an optimized **(O(log n)) time complexity** by halving the searchable elements with each iteration.

Table 6-3a. Binary Search for Arrays

Code

```
84  template <typename T>
85  Node<T>* getMiddle(Node<T>* start, Node<T>* last){
86
87      Node<T>* slow = start;
88      Node<T>* fast = start;
89
90      while(fast != last && fast->next != last){
91          slow = slow->next;
92          fast = fast->next->next;
93      }
94      return slow;
95  }
96
97  // ----------------------------------------- //
98  /* BINARY SEARCH LINKED LISTS */
99
00  template <typename T>
01  void binarySearhLL(Node<T>* start, T dataFind){
02
03      Node<T>* last = nullptr;
04
05      while (start != last) {
06          Node<T>* middle = getMiddle(start, last);
07
08          if(middle->data == dataFind){
09              std::cout << "The data was found.\n";
10              return;
11          }
12          else if(middle->data > dataFind){
13              last = middle;
14          }
15          else {
16              start = middle->next;
17          }
18      }
19      std::cout << "Search element was not found.\n";
20  }
```

Output

```
-----------------------------------
Search element was not found.
Search element was not found.
```

Observations

As for the second binary search algorithm, This code representation is the same searching algorithm, focuses on binary search algorithm, Using Linked List. Using the previous knowledge, a Two-Pointer Technique for identifying the midpoint, the implementation utilizes a slow and fast pointer approach getMiddle function to locate the center node of a specific sub-list segment. By moving the fast pointer two nodes at a time and the slow pointer one node at a time, the middle

	element is accurately found between the defined **start** and **last** boundaries. Unlike array-based binary search algorithms, which entirely use index arithmetic, this algorithm shifts pointer nodes. The search range narrows dynamically by either setting the **last** boundary to the **middle** node (discarding the right half) or shifting the **start** boundary to **middle->next** (discarding the left half) based on comparisons with the target data.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Code Snippet

[Main CPP of Linear Search Array]

```
    int targetKey = 18;
    int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int totalElements = sizeof(arr) / sizeof(arr[0]);
    int maxIndex = totalElements - 1;

    std::cout << "=== | Array Sequential Search | ===" << std::endl;
    linearSearchArr(maxIndex, targetKey, arr);
```

[Header Function of Linear Search Array]

```
// ---------------------------------------- //
/* LINEAR SEARCH ARRAY */

template <typename T>
void linearSearchArr(int maxSize, T dataFind, T data[]){
    int i = 0;
    int comparisons = 0;
    while(i <= maxSize){
        comparisons++;
        if(dataFind == data[i]){
            std::cout << "Searching is successful\n";
            std::cout << "Number of comparisons: " << comparisons << "\n";
            return;
        }
        i++;
    }
    std::cout << "the search is unsuccessful\n";
}
```

[Main CPP of Linear Search Linked List]

```
    std::cout << "\n=== | Linked List Sequential Search | ===" << std::endl;
    Node<int>* head = new_node(arr[0]);
    Node<int>* current = head;
    for (int i = 1; i < totalElements; ++i) {
        current->next = new_node(arr[i]);
        current = current->next;
    }
    linearSearchLL(head, targetKey);
```

[Header Function of Linear Search Linked List]

```
38  // ---------------------------------------- //
39  /* LINEAR SEARCH LINKED LIST */
40
41  template <typename T>
42  void linearSearchLL(Node<T> *node, T dataFind) {
43      int comparisons = 0;
44      while(node != nullptr){
45          comparisons++;
46          if(node->data == dataFind){
47              std::cout << "the search was successful.\n";
48              std::cout << "Number of comparisons: " << comparisons << "\n";
49              return;
50          }
51          node = node->next;
52      }
53      std::cout << "the search was unsuccessful\n";
54  }
```

Output Console

```
=== | Array Sequential Search | ===
Searching is successful
Number of comparisons: 2

=== | Linked List Sequential Search | ===
the search was successful.
Number of comparisons: 2
```

Problem 1 Question Analysis

In the code interpretation using the sequential search of the constant list, In both linked list and array approach, both have read a total of 2 comparisons necessary to find the key '18' in the const list. The sequential search algorithm iterates one by one and compares each data one by one if the key is equivalent to the target data that is stored within the target variable which is known as the **dataFind**. It has the same code structure formation as the code implementation in the procedural output, in which we only have to identify how many comparisons it does in each iteration to reach its target value / data.

Problem 2

Code Snippet

[main CPP of Linear Search Counting Instance Occurrences]

```cpp
// ---------------------------------------- //
/* LINEAR SEARCH COUNTING REPEATING INSTANCES */

template <typename T>
void countOccurrencesArr(int maxSize, T dataFind, T data[]) {
    int count = 0;
    int comparisons = 0;
    int i = 0;
    while(i <= maxSize) {
        comparisons++;
        if(data[i] == dataFind) {
            count++;
        }
        i++;
    }

    if(count > 0) {
        std::cout << "Element found with " << count << " repeating instance(s).\n";
    } else {
        std::cout << "Element not found.\n";
    }
    std::cout << "Total full scan comparisons: " << comparisons << "\n";
}
```

[Header File Function of Linear Search Counting Instance]

```cpp
    std::cout << "\n=== | Counting All Instances (Array) | ===" << std::endl;
    countOccurrencesArr(maxIndex, targetKey, arr);

    // Memory clean up
    current = head;
    while (current != nullptr) {
        Node<int>* nextNode = current->next;
        delete current;
        current = nextNode;
    }
```

Output Console

```
=== | Counting All Instances (Array) | ===
Element found with 2 repeating instance(s).
Total full scan comparisons: 10
```

Problem 2 Analysis

In this problem analysis, in the header function named **countOccurrencesArr()**, it handles the definition of variables. First is the count variable. The **count variable is the variable that counts the amount of occurrences of the target data as it iterates all over the array list**. The **comparison variable** is the total iteration comparison that was compared as each iteration. In the while loop condition is the overall method that identifies if the data is equal to the target variable, the count variable will be incremented.

Problem 3

Code Snippet

[Main CPP of Binary Search Array]

```
41      std::cout << "\n=== | Array Binary Search | ===" << std::endl;
42
43      int targetValue = 0;
44      int dataset[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
45
46      for(int i = 0; i < 10; i++){
47          // dataset[i] = i;
48          std::cout << dataset[i] << " ";
49      }
50      binarySearchArr(8, dataset, 10);
51      return 0;
52  }
```

[Header File Function of Binary Search Array]

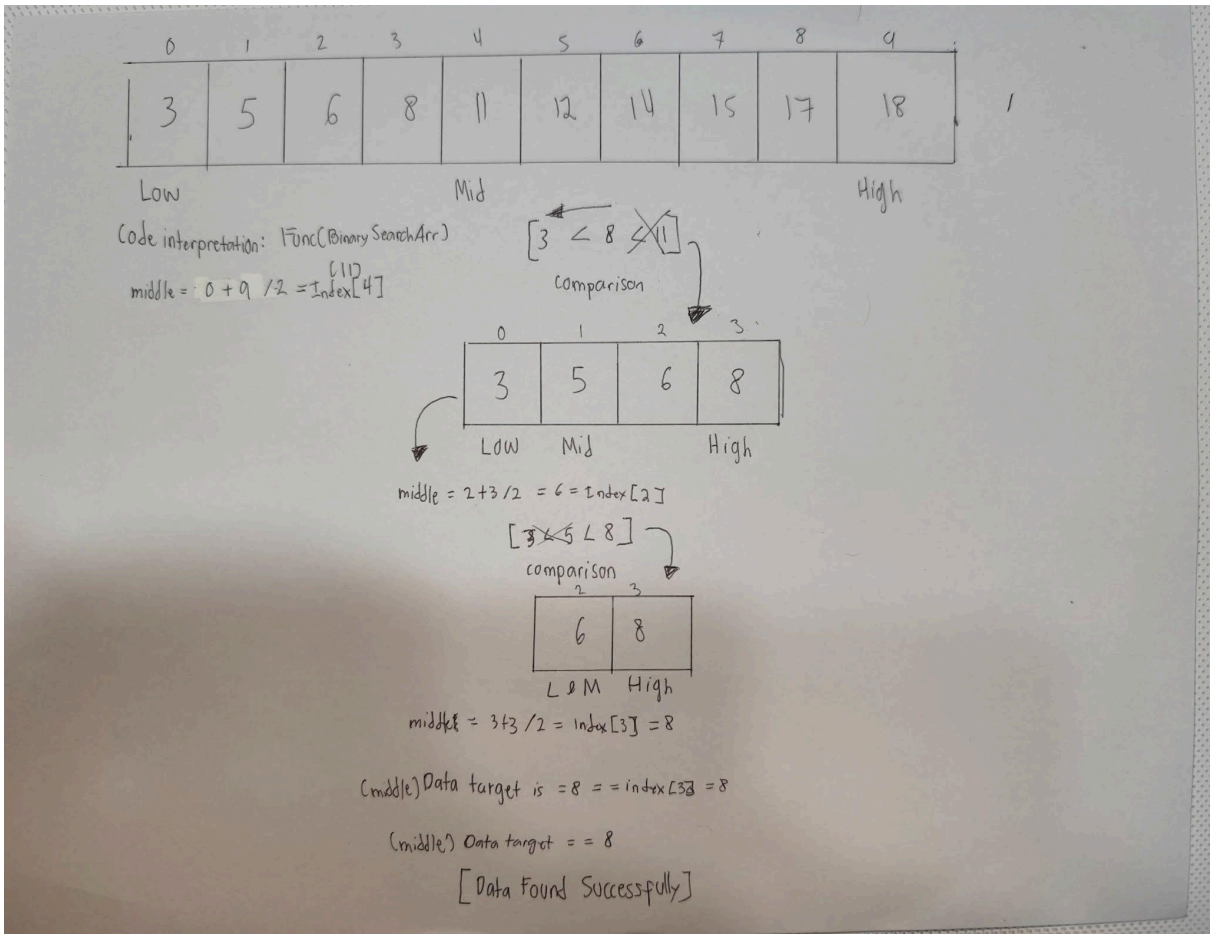
```
83  template <typename T>
84  void binarySearchArr(T dataFind, T data[], int maxSize){
85      int low = 0, up = maxSize-1, middle;
86      std::cout << std::endl;
87      while(low <= up){
88          middle = (int)((low+up)/2);
89
90          std::cout << "Low: " << low << " | Middle: " << middle
91          << " | High: " << up << "|" << std::endl;
92
93          if(dataFind == data[middle]){
94              std::cout << "\nSearching is successful\nValue [" << dataFind << "] Found.\n";
95              return;
96          }
97          else if(dataFind > data[middle]){
98              low = middle + 1;
99          }else {
100             up = middle - 1;
101         }
102     }
103     std::cout << "Search element was not found.\n";
104 }
```

Output Console

```
=== | Array Binary Search | ===
3 5 6 8 11 12 14 15 17 18
Low: 0 | Middle: 4 | High: 9|
Low: 0 | Middle: 1 | High: 3|
Low: 2 | Middle: 2 | High: 3|
Low: 3 | Middle: 3 | High: 3|

Searching is successful
Value [8] Found.
```

Diagram



Problem 3 Analysis

Using the diagram representation, it tracks how to function and process the middle and compares the data over the list of arrays to identify which side will be completely disregarded. The code computes for the middle to be the basis for finding the dataFind and comparison. As the list of arrays shortens, the middle gets closer and closer to the intended value to be compared for it to find the dataFind value for the program to successfully search its target value. Although in case that the list does not contain it, the program consists of cases to avoid bugs and crashes.

Problem 4

Code Snippet

```
43      std::cout << "\n=== | Array Recursive Binary Search | ===" << std::endl;
44
45      int dataset[maxSize] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
46
47      binarySearchArrRecursive(dataset, 0, maxSize-1, 8);
48
49      return 0;
```

```
83  template <typename T>
84  void binarySearchArrRecursive(T data[], int low, int up, T dataFind){
85      if(low > up){
86          std::cout << "Data key [ " << dataFind << " ] was not found.\n";
87      }
88
89      int middle = (int)((low+up)/2);
90
91      if(dataFind == data[middle]){
92          std::cout << "Searching is successful\nValue [" << dataFind << "] Found.\n
93          return;
94      }
95      else if(dataFind > data[middle]){
96          binarySearchArrRecursive(data, middle + 1, up, dataFind);
97      }else {
98          binarySearchArrRecursive(data, low, middle - 1, dataFind);
99      }
100 }
```

Output Console

```
=== | Array Recursive Binary Search | ===
Searching is successful
Value [8] Found.
```

Problem 4 Analysis

In this problem analysis, the definition of binary Search Recursive is that the function itself is the one calling her own methods until the program is done, or the target data value is found, or once the array list is iterated where it already reaches the end. As shown in the code representation above, the calling of function as it gets the parameter in the main driver file, it processes itself over and over again until a condition is met.

Conclusion

Overall, In this laboratory activity, I saw a lot of flaws, challenges and struggles while doing this activity. It has provided me with a wide and comprehensive understanding of how data structure fundamentals is important, especially on how efficiency is important in searching algorithms. I have learned a deep understanding between linear searching and binary searching. As linear searching has no strict requirements in the orders of data generated, it is very effective for unorganized datasets. It operates in O(N) time on both application of array and linked list. Meanwhile, binary search is interconnected to singly linked lists, it runs in a logarithmic time complexity O(log N) on arrays. Binary has strict requirements for it to be applicable for certain programs. The data should be in a sorted order for the binary to work in its very efficient way. There's more flaws and adjustments that are needed for me to improve. I struggled a lot to the point that I couldn't maintain my time management integrated with the pressure for this activity resulting in a late submission. Although it was able to help me gain more knowledge regarding searching algorithms and their different applications. There's more knowledge to learn and to explore, thus I need to do my best to successfully carry myself when the day comes that I'm struggling in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.